

Final Case Study

Tyler Bibus

Introduction

This case study will be focused on AI Training Data as a whole, with a specific focus on the controversial way that GitHub CoPilot – a tool created by Microsoft – collected and utilized its training data. Modern AI coding assistants are used everywhere, from universities to corporations. These models are theorized to boost efficiency and innovation by enabling programmers to write code much faster. These AI models are trained using mind-boggling amounts of data, and this data largely comes from publicly available places on the internet. The concern is that this training data has licensing to protect the intellectual property of the developers.

Background

Microsoft released their own Large-Language Model (LLM) powered coding assistant Copilot in June 2021. Copilot's training is specifically pulled from publicly available GitHub repositories. The licenses used by these repositories require credit and that their work be released as open source – the GPL license being one example. However, when copilot effectively copies snippets of code from these repositories, they do not provide attribution nor does its open source its own code. This is a potential violation of these licenses, raising ethical concerns over free use and intellectual property.

There are several stakeholders involved in this controversy: the developers of the open-source software, corporations like Microsoft and OpenAI, users of these coding assistants, and open-source software as a whole – this is effectively similar to “society” but specified down.

The developers are concerned that their intellectual property rights are being violated by the corporation who use their work to train their models. They feel that their work – and thus property rights – are being exploited in both an unethical and illegal way.

The corporations seek to train better models, which require more data to train. Corporations argue that for the sake of innovation they need to be able to have access to the developers' work. They also argue that they have a “Fair Use” exception due to the transformative nature of their work.

Users of these AI coding assistants might be unknowingly injecting copyrighted work into private software, raising legal concerns. They want the productivity boost from coding assistants but do not want to worry about potential ramifications from copyright violations. These users range from practicing engineers, to students, to other open-source developers.

Finally, the open-source software ecosystem – which is crucial to support platforms like Linux – require these licenses to be respected. When trust fails as social contracts collapse, and protective systems are rendered useless, open-source software development could be hindered.

Methods

To analyze this controversial and complex issue, I will be analyzing it through three different ethical frameworks.

The first ethical framework will be a deontological framework – specifically following the IEEE Code of Ethics as covered in class. This framework is very popular, used by many computer, software, and electrical engineers. Even in class, we cover many of the rules laid out by the IEEE code of ethics. There are key rules laid out that are relevant, but an important one is 1.5, “to seek, accept, and offer honest criticism of technical work, to acknowledge and correct errors, to be honest and realistic in stating claims or estimates based on available data, **and to credit properly the contributions of others;**” (IEEE). The key part that is bolded is especially important and will be referenced in the analysis section.

The second ethical framework will be a utilitarian consequentialist framework. This framework is possibly even more popular among engineers as it is very easy to get lost in the idea of seeking the best outcome all the time. I will be delving specifically into the potential implications of these coding assistants to justify whether – or whether not – these models are justified from this perspective.

The last ethical framework is effectively my own, I consider it a blend of consequentialism and deontology from a libertarian and natural rights perspective. I believe that my unique perspective of choosing the best outcome, if the means fall within the non-aggression principle, is a unique framework I have discussed in many previous reflections and papers.

A key ethical rule behind this framework is that property rights exist for scarce resources, and that ideas, patterns, and information are not scarce and thus not protected. This idea is largely derived from Hans-Hermann Hoppe, who argues in an interview that while ideas are goods, they are not scarce [rivalrous] goods (Kinsella). Code wouldn't fall

under the rivalrous requirement – effectively meaning that you can copy other peoples code without depriving them of being able to use it – as claiming ownership over abstract code and ideas would infringe on others’ rights to use their own property in a similar way, violating the non-aggression principle. An analogy of this is comparing the scarcity of a hammer and code. If I take your hammer, you are deprived of its capabilities and cannot build, if I copy your code, you are still able to use your code unimpeded.

Analysis

IEEE Code of Ethics Framework

To analyze from an IEEE code of ethics perspective, there are two main concerns that are raised: Did Microsoft violate canon 1.5 by stripping license headers from code during training, this violating the work and the concern of an engineer using copilot could have unknowingly injected GPL protected code, violating a license – also violating canon 1.5).

The first concern is if Microsoft violated canon 1.5 of the IEEE code of ethics when preparing code and training the models. This is a direct violation of the code of ethics, as using their work, even for training, without proper accreditation falls directly in failing to credit the contribution of others. The argument that training is transformative and thus protected **legally** might be true, but the law is not a perfect reflection of ethical practices. If you created a software tool that is later used by a movie you would be expected to be credited as your contribution led to a direct improvement in that movie. This analogy applies well to AI training data, as while you might not be able to directly point to what was improved with your work, your work used as training data directly contributed to the development and improvement of the model. From a legal perspective, in Doe 1 vs GitHub lawsuit, the district judge dismissed claims regarding their stripping of license headers because the output from the AI model was not identical. (Doe 1)

The second aspect of Microsoft’s potential violation of IEEE’s code of ethics is that their model could be injecting copyrighted code into others code without the developer knowing – thus leading to a conflict where credit is not given properly. While this is an easy assumption to make as true, the Doe 1 vs GitHub lawsuit showed that developers were unable to recreate exact 1:1 copies of the original copyrighted code. (Doe 1) The lawsuit has been narrowed down to a breach of contract rather than copyright claims. Licenses like the MIT license are legally binding, meaning that by ignoring the requirements of the license – proper attribution – they breach professional obligations and contracts. Because this is a breach of contract, it is unlawful and breaks canon 1.4 of the IEEE code of ethics, “to avoid

unlawful conduct in professional activities...” (IEEE). This lawsuit is still technically ongoing, so GitHub hasn’t been found guilty yet, but this appears to be a breach of contract.

Assuming Microsoft is found guilty in violation of these contracts, they should be forced to credit all developers that have the relevant licenses. Another way to remedy this would be to have them open source their code, giving back to the developer community in a way. That wouldn’t completely absolve them, as they would likely be legally obligated to provide settlements with the developers that they broke contract with.

Utilitarian Consequentialist Framework

The consequentialist framework weighs the pros and cons of the output of the situation. The usage of training data led to the creation of LLMs, specifically the coding models, which has been a massive boost in productivity also has potential to do harm to the open-source community.

The benefit of an AI coding assistant is that it can offer an incredible boost in productivity. Users of this model could see their output increase, both in volume and quality. This improvement in code quality could lead to an increase in the quality of life for the average person as well. These users now are often reliant on these tools and taking them away could be problematic for both users and companies alike.

Violating these licenses, however, could cause negative impacts on open-source software. These impacts could be as simple as the violation of these licenses without any retribution effectively nullifying them. If developers lose all rights to their work and anyone can use them, many developers might be incentivized to stop developing open-source software.

The question is, do the productivity benefits outweigh potential harms to open-source software. As open-source software is free, it ultimately gets funding through other means than just direct payment like proprietary software. Ignoring current outcomes, we can assume from these corporate points of view that them using their code as training data – not just copying and selling direct copies – doesn’t inherently affect these open-source developers as much as it helps current engineers. The benefits of the assistant can be much more easily quantified and felt, compared to the theoretical impacts that are secondary or tertiary effects from licenses being legally weakened.

A utilitarian perspective would advocate maximizing good. These companies could offer some amount of compensation to the developers, if it doesn’t impair their ability to innovate. If you ignore hindsight and look at it from an engineer working on Copilot you would want to develop the tool, as direct impacts are good, and potential secondary or

tertiary impacts are mixed, overall maximizing good would incentivize you to develop this tool.

Non-Aggression Principle Framework

As listed earlier in the methods section, this is my own framework I have been developing/discovering during the semester. This framework seeks the best outcome if it aligns with the Non-Aggression Principle as well as general libertarian thought. I hope my topic is unique, however, if it's a relatively common topic I hope my unique framework will be suitable.

To examine if this case violates my framework, we should first analyze the desired outcome, as well as the steps that were taken to get to the desired outcome. The outcome that Microsoft intended was to create a well-trained and accurate coding assistant. Whether their goal was to automate away jobs or boost productivity isn't exactly clear, so we will assume that they were only trying to create this model. This outcome doesn't inherently violate the non-aggression principle or natural rights for anyone.

Now we can analyze how they achieved their outcome, taking publicly available codebases and stripping off the licenses for training. From a non-aggression perspective, intellectual property generally isn't recognized due to its non-scarce nature – you cannot force someone to not think of something – however, because these licenses had contractual clauses Microsoft likely violated these contracts. Contract violation and voiding is illegal and would mean that the developers would be able to sue for damages. Damages could come from lost recognition leading to a loss as a job opportunity or sponsorship. In this way Microsoft directly violated the framework in their approach to reaching their outcome.

A good way to remedy this ethical issue would be to have Microsoft settle with these developers, as the model has already been trained. To prevent future issues like this, AI trainers should be required to follow the license. This could provide a potential incentive for both parties to agree upon a shared contract that, for some agreed upon compensation, their code is permitted to be used in the training data.

Recommendation

Across all three frameworks, Microsoft is wrong for violating intellectual property rights and our contracts with the open-source developers. We will analyze these recommendations that were derived from the different frameworks or are options that are currently being implemented.

The first resolution would be the status quo. This option would mean that there would be no action against Microsoft and other large companies. This would likely have large detrimental impacts on open-source software but also promises the most amount of AI “innovation”. Unlimited training data does lead to better models, but this does violate the contracts imposed on these companies by the licenses of the software. This leaves a legal hole in contract law specific to software.

The second resolution would be a license that allows developers to opt in to having their models trained. This could include royalties or just permission from the developers that buy into these licenses. This would respect intellectual property, satisfying the IEEE code of ethics. However, this recommendation could make training super expensive, slowing innovation and potentially leading to a worse outcome from a utilitarian perspective. Forcing people into these licenses also would violate the non-aggression principle, as intellectual property isn’t protected as strictly as rivalrous property.

The third resolution would be to implement some kind of attribution filter options. This option would require AI models to cite their sources, as well as increased focus on preventing exact code output. This would allow the models to exist inside the current license system, as viewing code and using ideas from it doesn’t violate the license – just using code ideas – the AI model would be no different than one person inspecting a GitHub repo and pulling some ideas from it without copying code. Even when using very similar code is required, the AI model would be able to provide citations, effectively allowing it to abide by the license issues.

Overall, the third recommendation, attribution and filtering, appears to be the most attractive option from our ethical framework perspectives. This would align well across the different frameworks, as it would allow proper credits given – satisfying deontological frameworks following IEEE COE – not be stifling AI innovation by making training prohibitively expensive or impossible – satisfying a utilitarian perspective – and not using government to impose these rules allows the non-aggression principle to be used correctly. This also effectively solves all current and potential issues the stakeholders have. Developers are getting credit, companies have training data, users have the model, the open-source software community is reasonably protected from collapsing due to license abuse.

Works Cited

IEEE. "IEEE Code of Ethics | IEEE." *ieee.org*, 2020, www.ieee.org/about/corporate/governance/p7-8.

Kinsella, Stephan. "Hoppe on Intellectual Property." *C4SIF.org*, 27 Dec. 2010, c4sif.org/2010/12/hoppe-on-intellectual-property/. Accessed 14 Dec. 2025.

Doe 1 v. GitHub, Inc., Order Granting in Part and Denying in Part Motions to Dismiss, Case No. 4:22-cv-06823-JST (N.D. Cal. May 11, 2024).